PHASE-II PROJECT PROGRESS

Project-Based Learning (PBL) | Academic Session 2026–27

CLASS PILOT

Your Smart Class Attendance Assistant

PROJECT AND TEAM INFORMATION

Project Detail	Information
Project Title	Class Pilot: Your Smart Class Attendance Assistant
Domain	Data Structures & C++
Department	Computer Science & Engineering
Institution	Graphic Era Deemed to Be University, Dehradun
Team Name	Class Pilot Team
Team ID / Team #	DSCPP-III-2026-T340 / T340
Course Name and ID	Data Structures & C++
Project Phase	Phase-II Project Progress

Project focus: a modular C++ attendance-planning application.

PROJECT TEAM

Team members listed in the existing Class Pilot project report:

Role / Member	Name and Student ID	Section	Email / Photo
Team member 1	Siddhant Gupta – 2510011681	C	sidgupta650@gmail.com
Team member 2	Abheer Sharma – 2510011271	G	Abheersharma99@gmail.com
Team member 3	Simran Bhagat – 2510012722	K	simranbhagat0912@gmail.com
Team member 4	Radhika Aggarwal – 2510011743	H	radhikagg770@gmail.com

Mentor

Mr. Kireet Joshi

Project summary

Class Pilot is intended to help students track attendance across subjects, understand the effect of planned absences, identify subjects that need attention, and correct a recently entered attendance status within a defined 48-hour period. The project uses C++ and standard data structures to demonstrate a modular approach to a practical student problem.

Team responsibilities should be confirmed by the team before final submission. The progress report below describes the current shared project state rather than claiming unverified individual contributions.

PHASE-II PROJECT PROGRESS DESCRIPTION

1. Project Abstract (2 pts)

Class Pilot is a student attendance assistant designed to make attendance tracking and planning easier. It stores subject-wise class totals, attended classes, and the required attendance percentage. The C++ application calculates the current percentage and reports whether a subject is safe or below its requirement. It also records new Present/Absent entries with timestamps and allows one correction to an individual entry during the first 48 hours. A what-if calculator estimates the effect of future absences and, when attendance falls below the requirement, estimates how many consecutive classes need to be attended to recover. A priority queue ranks subjects by attendance risk, while a FIFO queue manages upcoming classes. The C++17 console implementation is separated into modules so each responsibility can be tested and explained independently. Data is currently held in memory and is not persisted in a database. The system is a planning aid and is not intended to replace an institution's official attendance records.

2. Updated Project Approach and Architecture (2 pts)

The implementation is organized as a modular C++17 console application. ClassPilot.h declares the shared structures and functions. Subject.cpp manages subject details and attendance summaries; Attendance.cpp records individual class outcomes and timestamps; Correction48Hours.cpp applies the correction time limit; WhatIf.cpp calculates projected attendance; RiskAnalysis.cpp builds a priority queue; UpcomingClasses.cpp manages a FIFO queue; and main.cpp provides the menu and control flow. The main data structures are vector, unordered_map, queue, and priority_queue from the C++ Standard Library. Time checks use time_t, time(), and difftime(). The current application keeps its data in memory, so information is not retained after the program closes. A possible future improvement is to add persistent storage, for example with SQLite, after defining the data model and testing save/load behaviour.

TASKS, CHALLENGES AND PENDING WORK

3. Tasks Completed (7 pts)

The following tasks are part of the project, and all team members have contributed to the work. Since the available information does not clearly specify who was responsible for each task, the contributions have been considered a shared effort among the team members.

Task Completed	Team Member
Discussed and finalized the main features of the project, including attendance tracking, absence planning, attendance correction, risk assessment, and upcoming class details.	Siddhant Gupta
Organized the C++17 program into separate source files and created the common header file, ClassPilot.h, to manage shared declarations	Abheer Sharma
Developed the features for adding subjects, validating user inputs, calculating attendance percentages, and displaying attendance status as SAFE or SHORTAGE.	Siddhant, Abheer, Radhika
Added a feature to record students as Present or Absent for individual classes, along with a one-time correction option available within 48 hours.	Siddhant, Abheer, Radhika
Developed the option to check how future absences could affect attendance, rank attendance risks using priority_queue, and manage upcoming classes through queue operations.	Siddhant, Abheer, Radhika
Compiled and tested the different C++ source files together using the C++17 compiler settings to ensure the application could run as a single program.	Radhika Aggrawal, Simran Bhagat

4. Challenges / Roadblocks (7 pts)

A key challenge is keeping attendance totals consistent when a previously marked Present/Absent record is corrected. The current implementation updates the subject's attended count when a valid correction is made and prevents a second correction. The 48-hour check depends on the computer's local clock, so time settings must be reasonable. The application stores information in memory; therefore, data is lost when the C++ program closes. Historical totals entered when a subject is first created are aggregate values and do not have individual timestamps, so only newly marked records can be corrected. Future work includes adding persistent storage such as SQLite, validating edge cases, and testing the correction boundary at just under and just over 48 hours. The team should also confirm individual task ownership and publish the source repository.

5. Tasks Pending (7 pts)

Task Pending	Team Member (to complete)
Connect the project to an SQLite database to store subject details, attendance records, correction status, and upcoming class information.	The team will decide who is responsible for this task.
Check whether the attendance correction feature works correctly, including cases where the 48-hour time limit has expired or a record has already been corrected.	The team will assign a member to handle this task.
Test the program with different inputs, such as invalid values, empty subject names, duplicate subject names, and attendance percentages at the minimum and maximum limits.	Responsibility for this task will be decided by the team.
Create or update the project's source-code repository and include the repository link, branch name, and commit history for reference.	The team will discuss and allocate this task among its members.
Add the official course ID and, if required, include team members' email addresses and photographs. Also, confirm each member's actual contribution to the project.	The team will finalize the member responsible for completing this task.

OUTCOMES AND PROGRESS OVERVIEW

6. Project Outcome / Deliverables (2 pts)

The planned deliverable is a student attendance assistant that calculates subject attendance, identifies shortages, estimates the effect of future absences, provides a 48-hour correction period for newly created individual records, ranks attendance risk, and manages upcoming classes. The current deliverable is a modular C++17 console codebase. The code demonstrates the use of vectors, hash-based lookup, a FIFO queue, a priority queue, modular functions, and timestamp-based checks. Persistent storage remains a possible future improvement. The application should be described as a functional console prototype with in-memory data.

7. Progress Overview (2 pts)

The core application is implemented and compiles as a multi-file C++17 program. Subject management, attendance calculation, marking attendance, the correction rule, what-if planning, risk ranking, and upcoming-class operations are present. Remaining work includes expanding test coverage, considering persistent storage, and publishing the source repository. The current work is best described as a functional C++ console prototype. No schedule baseline was supplied, so ahead-of-schedule or behind-schedule status cannot be measured reliably.

Current progress snapshot

Area	Current Status	Notes
C++ console modules	Implemented; compilation checked	Data is held in memory
48-hour correction	Implemented for new individual records	Historical aggregate totals have no session timestamps
SQLite persistence	Pending	Needed to preserve data between runs
Automated/manual test evidence	Partial / to expand	Record actual test results before final submission

CODEBASE, TESTING AND DELIVERABLES

8. Codebase Information (2 pts)

Repository link: not yet provided / not yet confirmed.

Current code location: local project folder containing the C++ source files ClassPilot.h, Subject.cpp, Attendance.cpp, Correction48Hours.cpp, WhatIf.cpp, RiskAnalysis.cpp, UpcomingClasses.cpp, and main.cpp.

Build command used:

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp Subject.cpp Attendance.cpp Correction48Hours.cpp WhatIf.cpp
RiskAnalysis.cpp UpcomingClasses.cpp -o ClassPilot
```

Branch and important commits: not applicable until a repository is created or the team supplies its existing repository details. Before submission, add the actual repository URL, active branch, and commit identifiers; do not invent these values.

9. Testing and Validation Status (2 pts)

Test Type	Status (Pass/Fail)	Notes
C++17 compilation	PASS	Multi-file build completed with -Wall -Wextra -pedantic.
Subject creation and attendance percentage	Implemented; manual test evidence to record	Check zero classes and attended <= total.
Present/Absent marking	Implemented; manual test evidence to record	Each new mark creates a timestamped record.
48-hour correction	Implemented; boundary tests pending	Test eligible, expired, invalid ID, and repeated correction.
What-if absence calculation	Implemented; manual test evidence to record	Compare projected percentage and recovery count.
Priority-queue risk ranking	Implemented; manual test evidence to record	Confirm shortage subjects appear before safe subjects.
Upcoming-class queue	Implemented; manual test evidence to record	Confirm first-in, first-out ordering.

10. Deliverables Progress (2 pts)

Deliverable	Status	Current evidence / next step
Modular C++17 console application	Completed prototype	Source files and successful compilation.
Attendance and subject calculations	Implemented	Run manual cases and record results.
48-hour correction feature	Implemented for new entries	Add boundary and repeat-correction tests.
What-if calculator and risk ranking	Implemented	Validate with sample inputs.
Upcoming-class queue	Implemented	Validate FIFO behaviour.
SQLite database persistence	Pending	Save and reload project data.
Repository and commit history	Pending / details needed	Publish code and insert actual link/branch/commits.

Final status: The core C++ prototype is available. Expanded validation, optional database persistence, and repository details remain to be completed.